

JARVIS Voice Assistant

Post-Fix Code Audit Report

Comprehensive analysis of wake word system, voice pipeline, integration wiring, and remaining gaps after 27 claimed fixes.

Project: com.aistudio.jarvis.nxqpmw

Date: August 2025

Audit Type: Static Code Analysis

Scope: 12 source files, manifest, gradle configs

1. Executive Summary	4
1.1 Audit Scope and Methodology	4
1.2 Key Findings at a Glance	4
2. Verification of Previous Fixes	5
2.1 Verified Genuine Fixes	5
2.2 Fixes with Remaining Issues	6
3. New Critical Bugs Found	6
3.1 BUG-A: Wake Word Model is Fundamentally Wrong	6
3.2 BUG-B: Audio Buffer Size Mismatch Causes Sample Loss	7
3.3 BUG-C: ToneGenerator Resource Leak	7
3.4 BUG-D: AudioController Lifecycle Not Managed	8
3.5 BUG-E: Missing MediaPipe ProGuard Rules (Release Build Crash)	8
3.6 BUG-F: Potentially Non-Existent AGP Version	8
3.7 BUG-G: Gemini Text Responses Silently Ignored	9
3.8 BUG-H: Static Misleading Notification Text	9
3.9 BUG-I: userSpeakingFlow Never Emits False	9
4. Stubs, Placeholders, and Fake Implementations	10
4.1 Fake Wake Word: "Hey Jarvis" Does Not Exist	10
4.2 Phantom Phase 6: Proactive Intelligence System	10
4.3 Unused Dependencies: 20+ Libraries Never Referenced	10
4.4 Incomplete Settings UI	11
4.5 Dead Code and Unused Imports	11
5. Missing Functionality	11
5.1 No TTS Fallback for Text-Only Responses	11
5.2 No Conversation Transcript or History	12
5.3 No Text Message Support in Gemini Client	12
5.4 No Audio Record Retry Mechanism	12
6. Recommended Improvements	12

7. Priority Fix Code Snippets

13

7.1 Fix AudioController Lifecycle (BUG-D)

13

7.2 Fix ToneGenerator Leak (BUG-C)

13

7.3 Fix MediaPipe ProGuard Rules (BUG-E)

14

7.4 Fix AGP Version (BUG-F)

14

7.5 Fix Audio Buffer Mismatch (BUG-B)

14

1. Executive Summary

This report presents a comprehensive post-fix code audit of the JARVIS Voice Assistant Android project. The audit was conducted after 27 previously documented bug fixes were reportedly applied to the codebase. The primary focus areas are the Wake Word Detection System, the Voice Conversation System (Gemini Live), and the overall integration wiring between components. This assessment is intended to be brutally honest about the current state of the project, separating genuinely fixed issues from those that remain broken, partially addressed, or were never real problems to begin with.

The JARVIS project is an Android application that uses a foreground service to continuously listen for a wake word (intended to be "Hey Jarvis"), then establishes a real-time bidirectional WebSocket connection to Google Gemini 2.5 Flash for voice-based AI conversations. The system includes a floating overlay HUD that appears during active conversations, acoustic echo cancellation, local voice activity detection for barge-in support, and secure API key storage via Android Keystore. The architecture is sound in concept, but the execution has several critical gaps that prevent it from functioning as advertised.

1.1 Audit Scope and Methodology

Every source file in the project was read and analyzed line by line. This includes all Kotlin source files under `app/src/main/java/com/example/`, the `AndroidManifest.xml`, `build.gradle.kts`, `gradle/libs.versions.toml`, `proguard-rules.pro`, and all supporting configuration files. The audit cross-referenced the claimed fixes in `Voice_JARVIS_Bug_Fixes.md` against the actual code to verify each fix was genuinely applied. Special attention was given to runtime behavior that cannot be determined from static analysis alone, such as audio pipeline correctness, memory leak potential, and state machine transitions.

1.2 Key Findings at a Glance

Category	Count	Severity
Previously Fixed (Verified Genuine)	18	Resolved
Previously Fixed (Partially/Incorrectly)	5	Medium
Previously Claimed (Duplicate Entries)	4	Low
New Critical Bugs Found	9	High
Stub / Placeholder / Fake Code	5	High
Missing Functionality	4	Medium
Recommended Improvements	7	Low-Medium

2. Verification of Previous Fixes

The `Voice_JARVIS_Bug_Fixes.md` document claims 27 fixes were applied. However, this document contains significant duplication: items 13-17 appear twice (sections "Medium-Term Architecture Improvements" and "Medium-Term Architecture Improvements (Final Phase)" are repeated verbatim). After deduplication, there are 22 unique fixes. Of these, 18 are verified as genuinely applied in the current codebase. The remaining 4 have partial issues or were implemented with caveats that limit their effectiveness.

2.1 Verified Genuine Fixes

The following fixes have been confirmed as properly implemented in the current codebase through direct code inspection. Each item was verified by locating the exact code change described in the fix document and confirming it is present and correct in the current source files.

Fix #	Description	File Verified
01/03	Wake word labels changed to "yes"/"go" for speech_commands model	WakeWordEngine.kt:25
04	Audio buffer size corrected to 15600 samples	WakeWordEngine.kt:88-94
05	Resource leak fixed with nested use{} blocks	WakeWordEngine.kt:222-234
07	WebSocket onFailure respects isClosedManually flag	GeminiLiveClient.kt:151-155
08	consecutiveSendFailures uses AtomicInteger	GeminiLiveClient.kt:342
09	AudioTrack pause/flush/release in stopAudioIO	GeminiLiveClient.kt:396-406
10	AcousticEchoCanceller enabled on mic session	AudioController.kt:102-114
13	SettingsRepository converted to thread-safe Singleton	SettingsRepository.kt:21-28
06	Haptic + audio feedback on wake word detection	JarvisService.kt:198-212
33	READ_CONTACTS permission removed from manifest	AndroidManifest.xml
32	Lint abortOnError and checkReleaseBuilds enabled	build.gradle.kts:39-42
12/13	API Key sent via x-goog-api-key header (not query param)	GeminiLiveClient.kt:118
26	Single state source: JarvisServiceState object	JarvisService.kt:52-77
41	Overlay uses Material Icons (not emoji)	JarvisOverlay.kt:112-113
38	Architecture status cards use isServiceRunning flag	MainActivity.kt:514-524
36/37	Permissions flow requires core perms to proceed	PermissionsScreen.kt:240-253
17/31	R8 minification and ProGuard rules enabled	build.gradle.kts:24-31
40	Service state poll syncs via ActivityManager	MainActivity.kt:160-171

2.2 Fixes with Remaining Issues

While the above fixes are genuinely applied, several have subtle issues that limit their effectiveness or introduce new problems. These are not "broken" per se, but they represent areas where the fix was incomplete or where the implementation choice has downstream consequences that were not addressed.

Fix 09 (AudioTrack Lifecycle): The fix added pause/flush/release sequencing in `stopAudioIO()`. However, the release is dispatched to a coroutine (`clientScope.launch`) which means there is a race condition: if `destroy()` is called immediately after `stopAudioIO()`, the `audioTrack` reference is nulled before the coroutine runs, but the coroutine still holds a local reference (`trackToRelease`) so this is actually safe. The real issue is that `stop()` is never called on the `AudioTrack` before release. The `AudioTrack` documentation recommends calling `stop()` before `release()` to ensure the hardware buffer is fully drained. Currently, only `pause()` and `flush()` are called, which may cause an audible pop on some devices.

Fix 26 (Single State Source): The dual state management fix correctly removed `AssistantStateManager` and consolidated to `JarvisServiceState`. However, `JarvisServiceState` is a static Kotlin object with `MutableStateFlows`. This means its state survives process death. If the Android OS kills the service process (which is common for foreground services under memory pressure), and the user returns to the Activity, the static state will still show `isRunning=true` even though the service is dead. The polling fix (BUG-40) in `MainActivity` only runs when the Activity is in the foreground, so there is a window where the UI shows incorrect state.

Fix 40 (Service State Poll): The polling mechanism using `ActivityManager.getRunningServices()` works but has two drawbacks. First, `getRunningServices()` is deprecated on Android Q+ (API 29+). The code uses `@Suppress("DEPRECATION")` which silences the compiler warning but does not address the underlying deprecation. Second, polling every 1 second is wasteful. A better approach would be to observe the service lifecycle directly via a broadcast or to accept that static state may be stale and refresh it only on Activity resume.

3. New Critical Bugs Found

Despite the 27 claimed fixes, this audit has identified 9 new critical bugs that were not addressed in the previous fix rounds. These bugs range from resource leaks and lifecycle mismanagement to fundamental architecture mismatches that prevent the system from working as advertised. Each bug is described with its exact location, root cause, impact, and recommended fix.

3.1 BUG-A: Wake Word Model is Fundamentally Wrong

Severity: CRITICAL | File: WakeWordEngine.kt (line 25), JarvisService.kt (line 408)

The most significant finding of this audit is that the wake word system does not detect "Hey Jarvis" at all. The WakeWordEngine is configured with `targetLabels = setOf("yes", "go")` and uses the generic Google `speech_commands.tflite` model (a YAMNet-based audio classifier trained on 35+ keywords). There is no "hey_jarvis" label in this model. The code in `findAndLoadModelAsset()` searches for "hey_jarvis.tflite" first, but this file does not exist in the assets directory. Only `speech_commands.tflite` (3.6MB) is present. This means the system responds to the words "yes" and "go" instead of "Hey Jarvis". The notification text says "Listening for wake word 'Hey Jarvis'..." which is actively misleading to the user. The system will false-trigger on everyday conversation containing these common English words, making it unusable as a real wake word assistant.

Impact: The app cannot be used as a "Hey Jarvis" assistant. It triggers on common words, destroying user trust. This is the single most important issue to fix before any release.

Fix: Train a custom TFLite wake word model for "Hey Jarvis" using TensorFlow Lite Model Maker or Google's Audio Classification training pipeline. Alternatively, integrate a production wake word SDK like Picovoice Porcupine which supports custom wake words with proper false-reject/false-accept tuning. As a temporary measure, update all UI text and notifications to say "yes" or "go" instead of "Hey Jarvis" to be honest with users.

3.2 BUG-B: Audio Buffer Size Mismatch Causes Sample Loss

Severity: HIGH | File: AudioController.kt (line 121), WakeWordEngine.kt (line 133)

The AudioController captures audio in chunks of 1024 samples (line 121: `ShortArray(1024)`), while the WakeWordEngine accumulates these into a 15600-sample classification window (line 133: `ShortArray(15600)`). The problem is that $15600 / 1024 = 15.234$, which does not divide evenly. When the buffer has 15360 samples accumulated and a new 1024-sample chunk arrives, only $\min(1024, 15600 - 15360) = 240$ bytes are copied. The remaining 784 samples are silently discarded. This happens at every classification window boundary, creating a systematic 7.7% sample loss rate. This can cause missed wake word detections and degrades classification accuracy.

Fix: Either change the AudioController buffer size to a value that evenly divides 15600 (e.g., 780 samples, or 520 samples), or implement a proper ring buffer in WakeWordEngine that preserves overflow samples across window boundaries. The simplest fix is changing the AudioController buffer to 780 samples, since $15600 / 780 = 20$ exactly.

3.3 BUG-C: ToneGenerator Resource Leak

Severity: MEDIUM | File: JarvisService.kt (line 207-208)

The `playWakeWordFeedback()` method creates a ToneGenerator instance and starts a tone, but never calls `release()` on it. ToneGenerator holds native audio resources (an underlying AudioTrack). Each invocation leaks these resources. If the wake word is detected multiple times during a session (which is the expected usage pattern), each detection creates a new ToneGenerator that is never cleaned up. On

Android, native audio resources are limited, and leaking them can eventually cause other audio components (like the Gemini voice playback) to fail to initialize.

Fix: Call `toneGen.release()` in a finally block immediately after starting the tone, or use a shared `ToneGenerator` instance that is created once and reused throughout the service lifetime.

3.4 BUG-D: AudioController Lifecycle Not Managed

Severity: HIGH | File: JarvisService.kt (onDestroy), AudioController.kt

The `JarvisService.onDestroy()` method destroys the `WakeWordEngine` and disconnects the `GeminiLiveClient`, but it never calls `AudioController.destroy()`. This means that when the service is stopped by the user, the `AudioController`'s internal `CoroutineScope`, recording job, and `AudioRecord` instance remain alive. The `AudioRecord` will continue capturing audio in the background, consuming microphone hardware and battery. Since `AudioController` is a static Kotlin object (singleton), its scope and jobs persist independently of the service lifecycle. The `AudioRecord` will only stop when the state observer job detects that `isRunning` has been set to false, which creates a small window where the mic is still recording after the service is supposedly destroyed.

Fix: Add `AudioController.destroy()` call at the beginning of `JarvisService.onDestroy()`, before any other cleanup. This ensures the microphone is released immediately when the service is stopped.

3.5 BUG-E: Missing MediaPipe ProGuard Rules (Release Build Crash)

Severity: HIGH | File: proguard-rules.pro, build.gradle.kts (line 76)

The app uses `com.google.mediarpipe:tasks-audio:0.10.14` for audio classification, but the ProGuard configuration only includes keep rules for TensorFlow Lite (`org.tensorflow.**`), OkHttp, and Coroutines. MediaPipe is a completely separate library from TensorFlow Lite, and its classes will be aggressively obfuscated by R8 in release builds. This will cause runtime crashes when the `WakeWordEngine` tries to create an `AudioClassifier`, because the class names and method signatures used by the MediaPipe native layer will no longer match. The fix document (BUG-17/31) claims minification was enabled, but the necessary ProGuard rules for the actual audio library being used were never added.

Fix: Add the following line to `proguard-rules.pro`: `-keep class com.google.mediarpipe.** { *; }`

3.6 BUG-F: Potentially Non-Existent AGP Version

Severity: HIGH | File: gradle/libs.versions.toml (line 2)

The version catalog specifies `agp = "9.1.1"` for the Android Gradle Plugin. As of mid-2025, the latest stable AGP version is in the 8.x range. AGP 9.x has not been released by Google. This will cause a build failure with a dependency resolution error. Similarly, Kotlin version "2.2.10" is specified, which may not exist or may not be compatible with the (non-existent) AGP 9.1.1. These version numbers suggest the project was either generated by an AI tool that hallucinated version numbers, or was set up for a future

Android Studio version that does not yet exist.

Fix: Change `agp` to a stable version such as "8.7.3" and verify Kotlin compatibility (e.g., Kotlin 2.0.21 or 2.1.0). Test the build on a real Android Studio installation to confirm dependency resolution succeeds.

3.7 BUG-G: Gemini Text Responses Silently Ignored

Severity: MEDIUM | File: GeminiLiveClient.kt (lines 232-248)

The `handleIncomingMessage()` method only processes `inlineData` with `mimeType` starting with "audio/pcm". If the Gemini model responds with text-only content (which happens when the model decides a response does not warrant audio, or when the API falls back to text for certain query types), the text content in `modelTurn.parts` is completely ignored. The user sees the overlay change to SPEAKING state and then back to LISTENING, but no audio plays and no text is shown. This creates a confusing experience where the user knows the AI responded but cannot perceive the response.

Fix: Add a handler for text parts in `modelTurn`. When a text part is received and no audio is playing, use Android's native TextToSpeech engine to speak the text as a fallback. Additionally, consider showing the text in the overlay UI or logging it for debugging.

3.8 BUG-H: Static Misleading Notification Text

Severity: LOW | File: JarvisService.kt (lines 405-413)

The foreground service notification is created once in `createNotification()` and never updated. It always says "Listening for wake word 'Hey Jarvis'..." regardless of the actual state. When the assistant is in an active Gemini conversation (LISTENING or SPEAKING state), the notification still claims the system is listening for the wake word. When an error occurs, the notification still shows the idle message. This violates Android's foreground service notification requirements, which state that the notification should accurately reflect the current state of the foreground service.

Fix: Create a method `updateNotification(state: AssistantState)` that builds a new `Notification` with text appropriate to the current state and calls `NotificationManager.notify(NOTIFICATION_ID, updatedNotification)`. Call this method from `updateAssistantState()`.

3.9 BUG-I: userSpeakingFlow Never Emits False

Severity: LOW | File: AudioController.kt (lines 138-146)

The `userSpeakingFlow` `SharedFlow` emits `true` when the VAD detects 3 consecutive speech frames, but it never emits `false`. This means that once barge-in is triggered during a single model response, it cannot be triggered again even if the user stops speaking and starts again. In practice, this is partially mitigated because the `JarvisService` checks if `assistantState == AssistantState.SPEAKING` before acting on the barge-in signal, and the state transitions prevent re-triggering. However, the flow contract is broken: consumers that collect this flow will only ever receive a single `true` emission per Gemini session, which

makes the flow useless for any future feature that needs continuous speaking/not-speaking state tracking.

Fix: Add logic to emit false when the RMS drops below the VAD threshold for a sustained period (e.g., 10 consecutive non-speech frames). This makes the flow a proper binary speaking/not-speaking signal.

4. Stubs, Placeholders, and Fake Implementations

This section documents areas of the codebase that are not genuine implementations but rather stubs, placeholders, or misleadingly documented features. These are distinct from bugs (which are broken implementations) in that they represent functionality that simply does not exist despite being presented as working.

4.1 Fake Wake Word: "Hey Jarvis" Does Not Exist

The most significant fake in the codebase is the wake word itself. The entire user-facing experience (notification text, permissions screen description, architecture status card, home screen description) promises "Hey Jarvis" wake word detection. In reality, the system uses Google's generic `speech_commands.tflite` model and responds to the words "yes" and "go". The `whathavedone.md` document acknowledges this is a "temporary workaround" but the UI does not reflect this reality. A user who installs this app expecting to say "Hey Jarvis" will be confused when saying random words like "yes" triggers the assistant. This is not just a missing feature; it is a fundamental mismatch between what the product claims to do and what it actually does.

4.2 Phantom Phase 6: Proactive Intelligence System

The `whathavedone.md` document describes a "Phase 6 - Pillar 3: The Proactive Intelligence System" that includes a Room Database for local memory context, `UsageStatsManager` integration, `WorkManager` heartbeat, and a scoring engine. None of these components exist in the codebase. The `PermissionsHelper.kt` has a `hasUsageStatsPermission()` method and the `AndroidManifest.xml` declares `PACKAGE_USAGE_STATS` permission, but neither is used anywhere. Room database dependencies are declared in `libs.versions.toml` but never referenced in `build.gradle.kts`. This entire feature set is a planning document that was never implemented, yet the manifest still requests the usage stats permission, which will flag privacy concerns during Play Store review.

4.3 Unused Dependencies: 20+ Libraries Never Referenced

The `gradle/libs.versions.toml` file declares over 40 libraries, but only about 15 are actually used in the app's `build.gradle.kts`. The following declared libraries have zero references in the actual build configuration: Room (runtime, ktx, compiler), CameraX (camera2, lifecycle, view, core), Coil (coil-compose), Retrofit (retrofit, converter-moshi, moshi-kotlin, moshi-kotlin-codegen), Play Services

Location, Firebase (BOM, AI, App Check, Firestore, Auth), AndroidX Credentials, GoogleID, Roborazzi, TensorFlow Lite (standalone, as opposed to MediaPipe which is used), KSP plugin, Secrets Gradle plugin, and Google Services plugin. This bloat increases Gradle configuration time, confuses developers about what the project actually uses, and the KSP and plugin declarations may cause build failures if the plugins are not installed.

4.4 Incomplete Settings UI

The SettingsRepository supports five configurable parameters: API Key, System Prompt, Model Name, Wake Threshold, and Session Timeout. However, the SettingsScreen UI only exposes the API Key input field. The System Prompt, Model Name, Wake Threshold, and Session Timeout are stored and read correctly in the repository, but the user has no way to change them through the UI. The SettingsViewModel only has saveApiKey() and testApiKey() methods. This means the user cannot customize the assistant's behavior, personality, model selection, or sensitivity without manually editing the EncryptedSharedPreferences (which requires root or a debug build). For a product that markets itself as a "JARVIS Configuration" screen, this is a significant gap.

4.5 Dead Code and Unused Imports

Several minor code quality issues remain. The Theme.kt file imports lightColorScheme but never uses it (the app is hardcoded to dark mode only). The darkTheme parameter in JarvisTheme composable is accepted but ignored. The hasUsageStatsPermission() method in PermissionsHelper is defined but never called. The getUsageStatsPermissionIntent() method is defined but never called. These are not functional bugs, but they add maintenance overhead and can confuse new developers working on the project.

5. Missing Functionality

Beyond the bugs and stubs described above, there are several categories of functionality that the project needs but does not have at all. These are not bugs in existing code, but rather gaps where code should exist but does not.

5.1 No TTS Fallback for Text-Only Responses

The voice system is 100% dependent on the Gemini Live API returning PCM audio data. If the API returns a text-only response (which happens for certain types of queries, or when the model decides audio is unnecessary), the response is silently dropped. There is no Android TextToSpeech fallback. The user has no way to know what the AI said. A robust voice assistant should have a TTS fallback that speaks any text response that does not come with audio, ensuring the user always receives the AI's response through the voice channel they expect.

5.2 No Conversation Transcript or History

The entire voice conversation happens in real-time over WebSocket and is never persisted. Once the Gemini session ends and the system returns to wake word mode, all conversation context is lost. There is no chat history, no transcript view, and no way for the user to review what was discussed. For a personal AI assistant, this is a significant usability gap. At minimum, the app should store recent conversation transcripts locally (using Room or similar) and provide a scrollable history view in the main activity.

5.3 No Text Message Support in Gemini Client

The GeminiLiveClient can only send audio data to the WebSocket. It has no capability to send text messages. The Gemini Live API supports sending text alongside or instead of audio (via the `realtimeInput.message` field), which would enable features like typing a query, sending a pre-formatted prompt, or injecting context into the conversation. Without this, the system is limited to voice-only interaction, which reduces its flexibility in scenarios where voice input is impractical (noisy environments, privacy concerns, etc.).

5.4 No Audio Record Retry Mechanism

If `AudioController.startRecording()` fails to initialize the `AudioRecord` (e.g., because another app is currently using the microphone), it sets `audioRecord` to null and returns silently. There is no retry mechanism, no error propagation to the UI, and no state update to inform the user or the service that audio capture has failed. The system will appear to be working (the overlay shows, the service is running) but no audio is being captured or processed. The wake word engine will never detect anything because it receives no audio data. A retry mechanism with exponential backoff, combined with a user-facing error notification, would make the system much more robust.

6. Recommended Improvements

This section outlines improvements that are not bugs per se, but would significantly enhance the quality, reliability, and user experience of the JARVIS application. These are prioritized by impact and implementation effort.

#	Improvement	Impact	Effort
1	Train a custom "Hey Jarvis" TFLite wake word model or integrate Porcupine SDK	Critical	High
2	Add <code>AudioController.destroy()</code> call in <code>JarvisService.onDestroy()</code>	High	Trivial
3	Fix <code>ToneGenerator</code> leak by calling <code>release()</code> after use	Medium	Trivial
4	Add <code>MediaPipe ProGuard</code> rules for release builds	High	Trivial

#	Improvement	Impact	Effort
5	Fix AGP and Kotlin versions to stable releases	High	Low
6	Add TTS fallback for text-only Gemini responses	High	Medium
7	Make notification text dynamic based on assistant state	Medium	Low
8	Add UI controls for System Prompt, Model, Threshold, Timeout	Medium	Medium
9	Remove 20+ unused dependencies from libs.versions.toml	Low	Low
10	Remove PACKAGE_USAGE_STATS permission (unused)	Low	Trivial
11	Implement audio buffer ring buffer to prevent sample loss	Medium	Medium
12	Add conversation transcript storage and history view	Medium	High
13	Add text message support to GeminiLiveClient	Low	Medium
14	Replace deprecated getRunningServices() with proper lifecycle observation	Low	Medium

7. Priority Fix Code Snippets

This chapter provides ready-to-apply code fixes for the most critical bugs identified in Chapter 3. These are designed to be directly copy-pasted into the relevant source files with minimal modification.

7.1 Fix AudioController Lifecycle (BUG-D)

In JarvisService.kt, add the AudioController.destroy() call in onDestroy():

```
override fun onDestroy() {
    super.onDestroy()
    com.example.audio.AudioController.destroy() // <--- ADD THIS LINE
    lifecycleRegistry.handleLifecycleEvent(Lifecycle.Event.ON_PAUSE)
    // ... rest of cleanup
}
```

7.2 Fix ToneGenerator Leak (BUG-C)

In JarvisService.kt, modify playWakeWordFeedback():

```
private fun playWakeWordFeedback() {
    try {
        val vibrator = getSystemService(VIBRATOR_SERVICE) as android.os.Vibrator
        // ... vibration code ...
        val toneGen = android.media.ToneGenerator(AUDIO_MANAGER_STREAM_MUSIC, 100)
        toneGen.startTone(TONE_PROP_BEEP, 150)
        toneGen.release() // <--- ADD THIS LINE
    } catch (e: Exception) { Log.e("JarvisService", "Feedback error", e) }
}
```

7.3 Fix MediaPipe ProGuard Rules (BUG-E)

Add to proguard-rules.pro:

```
# MediaPipe Tasks Audio (used by WakeWordEngine)
-keep class com.google.mediatype.** { *; }
```

7.4 Fix AGP Version (BUG-F)

In gradle/libs.versions.toml, change:

```
# Before (non-existent version):
agp = "9.1.1"
kotlin = "2.2.10"

# After (stable versions):
agp = "8.7.3"
kotlin = "2.0.21"
```

7.5 Fix Audio Buffer Mismatch (BUG-B)

In AudioController.kt, change the recording buffer size from 1024 to 780:

```
// Before:
val buffer = ShortArray(1024)

// After (780 divides 15600 evenly: 15600 / 780 = 20):
val buffer = ShortArray(780)
```